

# Planejamento Lógico TaskTracker - Pedro Rebello Borges de Barros

Bootcamp II · Etapa 1

---

## Sumário

---

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Sumário                                                            | 1  |
| 1. Apresentação e entendimento do problema                         | 3  |
| 1.1 Identificação                                                  | 3  |
| 1.2 O problema a ser resolvido                                     | 3  |
| 1.3 A solução proposta                                             | 3  |
| 1.4 O que fica fora do escopo (e por quê)                          | 4  |
| 2. Decomposição do problema e dados do sistema                     | 4  |
| 2.1 Entradas de dados                                              | 4  |
| 2.2 Saídas de dados                                                | 5  |
| 2.3 Modelo de dados                                                | 6  |
| 2.4 Regras do sistema                                              | 7  |
| 2.5 Casos de erro tratados                                         | 8  |
| 3. Receita lógica do sistema (algoritmo em português estruturado)  | 8  |
| Passo 0 — Preparação (acontece uma única vez, ao ligar o sistema)  | 8  |
| Passo 1 — Entrar no sistema                                        | 9  |
| Passo 2 — Exibir o painel de tarefas                               | 9  |
| Passo 3 — Cadastrar uma tarefa                                     | 10 |
| Passo 4 — Editar uma tarefa                                        | 10 |
| Passo 5 — Concluir e reabrir uma tarefa                            | 11 |
| Passo 6 — Excluir uma tarefa                                       | 11 |
| Passo 7 — Visualizar por período e buscar                          | 11 |
| Passo 8 — Gerar o relatório resumido                               | 11 |
| Passo 9 — Visão administrativa                                     | 12 |
| Passo 10 — Encerrar                                                | 12 |
| 4. Mapeamento arquitetural para Git e GitHub                       | 12 |
| 4.1 Árvore de pastas do repositório                                | 12 |
| 4.2 Propósito de cada pasta e arquivo                              | 13 |
| 4.3 Conceitos de Git e GitHub que serão aplicados na próxima etapa | 15 |
| 5. Link da apresentação em vídeo                                   | 16 |

# 1. Apresentação e entendimento do problema

## 1.1 Identificação

| Campo                | Conteúdo                                        |
|----------------------|-------------------------------------------------|
| Nome completo        | Pedro Rebello Borges de Barros                  |
| Curso                | Ciência de Dados e Machine Learning             |
| Polo                 | Campus Asa Norte                                |
| Turma                | A                                               |
| E-mail               | pedro.rebello@sempreceub.com                    |
| Disciplina           | Bootcamp II                                     |
| Repositório previsto | github.com/PedroRebello1/task-tracker-bootcamp2 |

## 1.2 O problema a ser resolvido

Hoje organizo minhas tarefas do mesmo jeito que quase todo mundo faz: um pouco na cabeça, um pouco em mensagens que mando para mim mesmo e um pouco num caderno. Funciona até o momento em que o volume cresce, e no meio de um semestre com várias disciplinas, estágio e compromissos pessoais, ele sempre cresce.

O problema não é esquecer de anotar. O problema é que uma anotação solta não responde às perguntas que realmente importam:

1. Não existe visão de prazo. Um caderno não sabe que dia é hoje. Para descobrir o que vence nesta semana eu preciso reler tudo o que escrevi e comparar data por data.
2. Não existe prioridade real. Numa lista de papel, "comprar pão" e "entregar o projeto do Bootcamp" ocupam exatamente a mesma linha, com o mesmo peso visual.
3. Não existe registro do que foi feito. Quando eu risco um item do papel, a informação desaparece. No fim do mês eu não consigo responder se dei conta do que precisava fazer, porque a evidência foi literalmente rabiscada.
4. A informação não se filtra sozinha. Papel não tem filtro, não tem busca e não tem ordenação. Se eu quiser ver só o que é de Estudo, ou só o que é prioridade Alta, o único jeito é ler tudo de novo.

Além disso, o fato de que, se a anotação está espalhada em quatro lugares diferentes, não existe uma fonte de verdade, e duas listas que discordam entre si são piores do que nenhuma lista.

## 1.3 A solução proposta

O Task Tracker é um sistema de gerenciamento de tarefas que roda como uma aplicação web. O programa é escrito em Python, com um servidor na própria máquina que o usuário acessa pelo navegador. Cada pessoa cria uma conta, faz login e passa a ter uma área própria, onde só enxerga e altera as suas próprias tarefas.

Na prática, o sistema resolve os quatro problemas acima assim:

- Visão de prazo: o painel principal tem filtros de período. Ao entrar, o usuário já cai no filtro "Hoje" e vê o que precisa fazer naquele dia.. O filtro "Semana" mostra tudo entre segunda e domingo.

- Prioridade: toda tarefa tem obrigatoriamente uma prioridade escolhida numa lista fechada: Alta, Média ou Baixa, e a listagem é ordenada por data e, dentro do mesmo dia, pela prioridade.
- Perda de histórico: concluir uma tarefa não a apaga, mas muda o status de "Pendente" para "Concluída".
- Falta de filtros: além dos filtros de período, existem categorias e uma busca por palavras-chave no título.

Existe também o perfil administrador, criado na primeira execução do sistema. O administrador tem acesso a uma visão na qual enxerga e edita as tarefas de todos os usuários, servindo demonstrar o sistema e para justificar, na prática, a regra de isolamento entre contas.

## 1.4 O que fica fora do escopo

Definir o que o sistema não faz é parte de entender o problema. Ficam de fora:

- Notificações (push, e-mail, alarme): exigiria uma maior complexidade, sempre rodando em segundo plano.
- Tarefas recorrentes: a lógica de geração e edição de séries é desproporcional ao tamanho da entrega.
- Anexos e compartilhamento de tarefas: fogem do objetivo localhost, e demandam bem mais tempo.

Essas exclusões são deliberadas. Prefiro um núcleo funcional a muitos recursos incompletos.

## 2. Decomposição do problema e dados do sistema

### 2.1 Entradas de dados

Tudo o que o usuário informa ao sistema, e como ele informa:

| Entrada                 | Tipo                       | Obrigatório | Como é informado                                                             |
|-------------------------|----------------------------|-------------|------------------------------------------------------------------------------|
| Nome do usuário         | Texto                      | Sim         | Campo de texto (tela de cadastro)                                            |
| E-mail (login)          | Texto                      | Sim         | Campo de texto, formato de e-mail                                            |
| Senha                   | Texto                      | Sim         | Campo de senha, mínimo de 6 caracteres                                       |
| Título da tarefa        | Texto (até 60 caracteres)  | Sim         | Campo de texto                                                               |
| Descrição da tarefa     | Texto (até 300 caracteres) | Não         | Área de texto                                                                |
| Data prevista           | Data                       | Sim         | Seletor de calendário, já limitado para não permitir datas anteriores a hoje |
| Prioridade              | Texto de lista fechada     | Sim         | Seletor com Alta / Média / Baixa (padrão: Média)                             |
| Categoria               | Texto de lista fechada     | Não         | Seletor com Estudo / Trabalho / Pessoal / Saúde / Geral (padrão: Geral)      |
| Filtro de período       | Opção                      | Sim         | Botões Hoje / Semana / Mês / Todas (padrão: Hoje)                            |
| Termo de busca          | Texto                      | Não         | Campo de busca por palavra no título                                         |
| Identificador da tarefa | Número inteiro             | Sim         | Vem do botão da linha (Editar / Concluir / Excluir)                          |
| Confirmação de exclusão | Opção (Sim / Não)          | Sim         | Botões na tela de confirmação                                                |

As entradas de data, prioridade, etc nunca são digitadas. Usar listas de opção elimina o erro de digitação, o que sustenta as regras RN02 e RN03.

## 2.2 Saídas de dados

Tudo o que o sistema devolve na tela:

| Saída                             | Quando aparece                                                                                                                        |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Tela de login / cadastro          | Ao abrir o sistema, ou após sair                                                                                                      |
| Mensagem de erro de autenticação  | E-mail ou senha inválidos; e-mail já cadastrado; senha curta                                                                          |
| Painel de tarefas                 | Após o login bem-sucedido                                                                                                             |
| Lista de tarefas                  | Sempre que o painel é exibido; colunas: ID, título, prioridade, categoria, data prevista, status                                      |
| Faixa de resumo                   | No topo do painel: total no período, pendentes, concluídas, atrasadas                                                                 |
| Mensagem de lista vazia           | Quando o filtro não retorna nenhuma tarefa                                                                                            |
| Formulário de tarefa              | Ao criar ou ao editar                                                                                                                 |
| Mensagens de validação            | Sempre que uma regra de negócio é violada — sempre indicando o campo e o motivo                                                       |
| Mensagem de sucesso               | "Tarefa criada", "Tarefa atualizada", "Tarefa concluída", "Tarefa excluída"                                                           |
| Tela de confirmação de exclusão   | Antes de qualquer exclusão, mostrando título e data da tarefa                                                                         |
| Relatório resumido                | Ao acionar "Relatório": total, concluídas, pendentes, atrasadas, percentual de conclusão, distribuição por prioridade e por categoria |
| Visão administrativa              | Somente para o perfil administrador: todas as tarefas de todos os usuários, com a coluna do dono                                      |
| Mensagem de acesso negado         | Ao tentar acessar tarefa de outro usuário ou a visão administrativa sem perfil de administrador                                       |
| Página de endereço não encontrado | Ao acessar um endereço que não existe no sistema                                                                                      |

## 2.3 Modelo de dados

### Tarefa

| Atributo       | Tipo              | Obrigatório | Origem               | Observação                                      |
|----------------|-------------------|-------------|----------------------|-------------------------------------------------|
| id             | Inteiro           | Sim         | Gerado pelo sistema  | Identificador único e sequencial                |
| título         | Texto             | Sim         | Usuário              | Até 60 caracteres, sem nulos                    |
| descricao      | Texto             | Não         | Usuário              | Até 300 caracteres                              |
| data_prevista  | Data (AAAA-MM-DD) | Sim         | Usuário (calendário) | Não pode ser anterior à data do cadastro        |
| prioridade     | Texto (seletor)   | Sim         | Usuário (seletor)    | Alta, Média ou Baixa                            |
| categoria      | Texto (seletor)   | Não         | Usuário (seletor)    | Estudo, Trabalho, Pessoal, Saúde, Geral         |
| status         | Texto (seletor)   | Sim         | Gerado pelo sistema  | Pendente ou Concluída                           |
| data_criacao   | Data e hora       | Sim         | Gerado pelo sistema  | Registrado no momento da criação                |
| data_conclusao | Data e hora       | Não         | Gerado pelo sistema  | Vazio enquanto Pendente; preenchido ao concluir |
| usuario_id     | Inteiro           | Sim         | Gerado pelo sistema  | Dono da tarefa                                  |

## Usuário

| Atributo | Tipo                  | Obrigatório | Observação                                      |
|----------|-----------------------|-------------|-------------------------------------------------|
| id       | Inteiro               | Sim         | Identificador único                             |
| nome     | Texto                 | Sim         | Nome de exibição                                |
| login    | Texto (e-mail)        | Sim         | Único no sistema; é o que se digita para entrar |
| senha    | Texto                 | Sim         | Guardada com hash, nunca em texto puro          |
| perfil   | Texto (lista fechada) | Sim         | admin ou comum                                  |

Persistência: dois arquivos no formato JSON. Cada operação que altera dados regrava o arquivo inteiro, de modo que nada se perde ao fechar o programa. O JSON foi escolhido por ser legível a olho nu, por ser nativo do Python e por se comportar bem como volume montado no Docker.

## 2.4 Regras do sistema

O sistema tem sete regras de negócio explícitas. Cada uma é verificada no servidor, mesmo quando a interface já a impede. A interface previne o erro, a verificação no servidor garante a integridade.

RN01 — O título é obrigatório e não pode ser vazio. Verificada na criação e na edição. Um título formado apenas por espaços é tratado como vazio. Resposta do sistema: "Informe um título para a tarefa." O formulário é reexibido com os demais campos preservados.

RN02 — A data prevista não pode ser anterior à data da criação. Barreira dupla: o seletor de calendário já nasce limitado e, ao receber o formulário, o sistema confere a data novamente. Resposta do sistema: "A data prevista não pode ser anterior a hoje (xx/xx/xxxx)."

RN03 — Prioridade e categoria só aceitam valores de uma lista fechada. Prioridade: Alta, Média e Baixa, Categoria: Estudo, Trabalho, Pessoal, Saúde e Geral. Na tela, ambos são seletores. Resposta do sistema: "Prioridade inválida. Os valores aceitos são Alta, Média ou Baixa."

RN04 — Uma tarefa concluída não pode ser editada. Para alterar qualquer campo de uma tarefa Concluída, é preciso primeiro reabri-la. Isso protege o histórico, uma conclusão registrada não é reescrita por acidente. Resposta do sistema: "Esta tarefa está concluída. Reabra-a para poder editar." - acompanhada do botão Reabrir, que devolve o status para Pendente e limpa a data de conclusão.

RN05 — Cada usuário só enxerga e altera as próprias tarefas; o administrador é a exceção. Toda operação sobre uma tarefa confere se o usuario\_id da tarefa é o da sessão. Quem tem perfil admin passa por essa verificação e enxerga tudo. Resposta do sistema: "Você não tem permissão para acessar esta tarefa." O mesmo vale para a visão administrativa: um usuário comum que tente acessá-la recebe "Acesso restrito ao administrador."

RN06 — Toda exclusão exige confirmação explícita. Nenhuma tarefa é removida no primeiro clique. O sistema abre uma tela de confirmação identificando a tarefa. Resposta do sistema: "Excluir a tarefa 'xyz', prevista para xx/xx/xxxx? Esta ação não pode ser desfeita." — com os botões Sim, excluir e Cancelar. Se o usuário cancelar, absolutamente nada é alterado no arquivo.

RN07 — A data de conclusão nunca pode ser anterior à data de criação. Verificada no ato de concluir. É a regra que mantém a coerência temporal do histórico. Resposta do sistema: "Não foi possível concluir: a data de conclusão seria anterior à data de criação da tarefa." A tarefa permanece Pendente.

## 2.5 Casos de erro tratados

Além das sete regras, o sistema trata explicitamente:

| Situação                            | Tratamento                                                                                                                                                      |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| E-mail ou senha inválidos           | Mensagem genérica ("E-mail ou senha inválidos"), sem revelar qual dos dois está errado. Após 3 tentativas seguidas, o formulário fica bloqueado por 60 segundos |
| E-mail já cadastrado                | "Já existe uma conta com este e-mail."                                                                                                                          |
| Senha com menos de 6 caracteres     | "A senha deve ter pelo menos 6 caracteres."                                                                                                                     |
| Título acima de 60 caracteres       | "O título deve ter no máximo 60 caracteres."                                                                                                                    |
| Data vazia ou em formato inválido   | "Selecione uma data válida no calendário."                                                                                                                      |
| Identificador de tarefa inexistente | "Tarefa não encontrada." e retorno ao painel                                                                                                                    |
| Filtro sem resultados               | "Nenhuma tarefa encontrada para este período."                                                                                                                  |
| Arquivo de dados inexistente        | Na primeira execução, o sistema cria os arquivos automaticamente — usuarios.json já com o administrador padrão e tarefas.json vazio                             |
| Arquivo de dados corrompido         | O sistema avisa que o arquivo não pôde ser lido e não o sobrescreve, para não destruir dados recuperáveis                                                       |
| Endereço inexistente                | Página "Página não encontrada", com link de volta ao painel                                                                                                     |

## 3. Receita lógica do sistema em português estruturado

Abaixo é descrito o comportamento do sistema em linguagem, sem código. Ele é a receita que será feita em python na Etapa 2.

### Passo 0 — Preparação (uma única vez, ao rodar o sistema)

1. O sistema procura a pasta de dados.
2. Se o arquivo de usuários não existir, o sistema cria um arquivo novo contendo um único usuário administrador padrão (nome "Administrador", perfil admin, com senha inicial definida na configuração).
3. Se o arquivo de tarefas não existir, o sistema cria um arquivo com uma lista vazia.
4. O sistema carrega os dois arquivos para a memória. Se algum deles estiver ilegível, ele avisa o usuário e interrompe a inicialização, sem sobrescrever nada.
5. O sistema sobe o servidor local e informa o endereço de acesso na tela.

### Passo 1 — Entrar no sistema

1. O sistema mostra a tela de login, com os campos de e-mail e senha e um link "Criar conta".
2. Se o usuário escolher criar conta:
  - a) Ele informa nome, e-mail e senha.
  - b) O sistema verifica, nesta ordem: se todos os campos foram preenchidos; se o e-mail ainda não está cadastrado; se a senha tem pelo menos 6 caracteres.
  - c) Se alguma verificação falhar, o sistema mostra a mensagem correspondente e mantém o usuário na tela, com o que já foi digitado.
  - d) Se todas passarem, o sistema grava o novo usuário com perfil comum, guardando a senha de forma embaralhada, e devolve o usuário à tela de login.
3. O usuário informa e-mail e senha e envia.

4. O sistema procura o e-mail na lista de usuários.
5. Se o e-mail não existir, ou se a senha não conferir, o sistema mostra "E-mail ou senha inválidos" e soma 1 à contagem de tentativas.
6. Se a contagem chegar a 3, o sistema bloqueia novas tentativas por 60 segundos.
7. Se conferir, o sistema abre uma sessão guardando quem entrou e qual é o perfil, zera a contagem de tentativas e segue para o painel.

## Passo 2 — Exibir o painel de tarefas

1. O sistema lê a lista completa de tarefas.
2. O sistema decide quais tarefas considerar:
  - a) Se o perfil é comum, considera apenas as tarefas cujo dono é o usuário da sessão.
  - b) Se o perfil é admin e a visão administrativa está ligada, considera todas as tarefas de todos os usuários.
3. O sistema aplica o filtro de período escolhido (o padrão, ao entrar, é Hoje):
  - a) Hoje — mantém as tarefas cuja data prevista é igual à data atual.
  - b) Semana — mantém as tarefas cuja data prevista está entre a segunda-feira e o domingo da semana atual.
  - c) Mês — mantém as tarefas cuja data prevista cai no mês e ano atuais.
  - d) Todas — não descarta nenhuma.
4. Se houver um termo de busca, o sistema mantém apenas as tarefas cujo título contém esse termo, ignorando maiúsculas e acentos.
5. O sistema ordena o resultado: primeiro pela data prevista, da mais próxima para a mais distante; havendo empate, por prioridade (Alta, depois Média, depois Baixa); havendo novo empate, pelo identificador, em ordem crescente.
6. Se a lista final estiver vazia, o sistema exibe "Nenhuma tarefa encontrada para este período." e oferece o botão de criar tarefa.
7. Se não estiver vazia, o sistema exibe a tabela de tarefas e, acima dela, a faixa de resumo com quatro números: total no período, pendentes, concluídas e atrasadas (atrasada = status Pendente e data prevista anterior a hoje).
8. O sistema aguarda a próxima ação do usuário: criar, editar, concluir, reabrir, excluir, trocar o período, buscar, abrir o relatório, abrir a visão administrativa ou sair.

## Passo 3 — Cadastrar uma tarefa

1. O usuário aciona "Nova tarefa".
2. O sistema abre o formulário com: título, descrição, um calendário já limitado a partir do dia de hoje, um seletor de prioridade com "Média" pré-selecionada e um seletor de categoria com "Geral" pré-selecionada.
3. O usuário preenche e envia.
4. O sistema verifica, nesta ordem:
  - a) O título está preenchido e não é apenas espaços? (RN01)
  - b) O título tem no máximo 60 caracteres?
  - c) A data foi informada e não é anterior a hoje? (RN02)
  - d) A prioridade é um dos três valores aceitos? (RN03)
  - e) A categoria é um dos cinco valores aceitos? (RN03)

5. Se alguma verificação falhar, o sistema exibe a mensagem específica daquele campo e reexibe o formulário preservando tudo o que já havia sido digitado.
6. Se todas passarem:
  - a) O sistema gera o identificador: o maior identificador mais um.
  - b) Define o status como "Pendente", carimba a data e a hora de criação, deixa a data de conclusão vazia e registra como dono o usuário da sessão.
  - c) Acrescenta a tarefa à lista e grava no arquivo de tarefas.
  - d) Exibe "Tarefa criada com sucesso." e volta ao painel.

#### Passo 4 — Editar uma tarefa

1. O usuário aciona "Editar" na linha de uma tarefa.
2. O sistema procura a tarefa pelo identificador. Se não encontrar, exibe "Tarefa não encontrada." e volta ao painel.
3. O sistema verifica a propriedade: se o usuário não é o dono e não é administrador, exibe "Você não tem permissão para acessar esta tarefa." e volta ao painel. (RN05)
4. O sistema verifica o status: se a tarefa está Concluída, exibe "Esta tarefa está concluída. Reabra-a para poder editar.", oferece o botão Reabrir e não abre o formulário. (RN04)
5. O sistema abre o mesmo formulário do Passo 3, já preenchido com os valores atuais.
6. O usuário altera o que quiser e envia.
7. O sistema repete as verificações do item 4 do Passo 3, com um ajuste na regra da data: se a tarefa já tinha uma data no passado, é permitido mantê-la como está; o que não é permitido é trocá-la por uma nova data anterior a hoje.
8. Se todas passarem, o sistema grava as alterações, regrava o arquivo, exibe "Tarefa atualizada com sucesso." e volta ao painel.

#### Passo 5 — Concluir e reabrir uma tarefa

1. O usuário aciona "Concluir" na linha de uma tarefa.
2. O sistema procura a tarefa e verifica a propriedade, exatamente como nos itens 2 e 3 do Passo 4.
3. Se a tarefa já estiver Concluída, o sistema avisa "Esta tarefa já está concluída." e não altera nada.
4. O sistema prepara a mudança: status passa a "Concluída" e a data de conclusão recebe a data e a hora atuais.
5. Antes de gravar, o sistema confere se a data de conclusão é igual ou posterior à data de criação. (RN07)
  - a) Se não for, exibe "Não foi possível concluir: a data de conclusão seria anterior à data de criação da tarefa." e mantém a tarefa Pendente.
6. Se a conferência passar, o sistema regrava o arquivo e exibe "Tarefa concluída."
7. Para reabrir: o usuário aciona "Reabrir"; o sistema verifica propriedade, devolve o status para "Pendente", limpa a data de conclusão, regrava o arquivo e exibe "Tarefa reaberta."

## Passo 6 — Excluir uma tarefa

1. O usuário aciona "Excluir" na linha de uma tarefa.
2. O sistema procura a tarefa e verifica a propriedade, como nos itens 2 e 3 do Passo 4.
3. O sistema não exclui nada ainda: abre uma tela de confirmação mostrando o título e a data prevista da tarefa e o aviso de que a ação não pode ser desfeita. (RN06)
4. Se o usuário escolher Cancelar, o sistema volta ao painel sem alterar coisa alguma.
5. Se o usuário escolher Sim, excluir, o sistema remove a tarefa da lista, regrava o arquivo e exibe "Tarefa excluída."

## Passo 7 — Visualizar por período e buscar

1. O usuário escolhe um dos botões de período: Hoje, Semana, Mês ou Todas.
2. O sistema guarda a escolha e repete o Passo 2 do item 3 em diante.
3. Se o usuário digitar um termo na busca, o sistema aplica o termo junto com o filtro de período — os dois se somam, não se substituem.
4. Se o usuário limpar a busca, o sistema volta a exibir todo o período selecionado.

## Passo 8 — Gerar o relatório resumido

1. O usuário aciona "Relatório".
2. O sistema considera exatamente o mesmo conjunto de tarefas que o painel está exibindo (mesmo dono, mesmo período, mesma busca).
3. Se o conjunto estiver vazio, exibe "Não há dados suficientes para gerar o relatório neste período."
4. Caso contrário, o sistema calcula: total de tarefas; quantas estão Concluídas; quantas estão Pendentes; quantas estão atrasadas; o percentual de conclusão; a distribuição por prioridade; e a distribuição por categoria.
5. O sistema exibe os números e oferece o botão de voltar ao painel.

## Passo 9 — Visão administrativa

1. O usuário aciona "Visão administrativa".
2. Se o perfil não for admin, o sistema exibe "Acesso restrito ao administrador." e volta ao painel. (RN05)
3. Sendo administrador, o sistema repete o Passo 2 considerando todas as tarefas de todos os usuários e acrescenta à tabela uma coluna com o nome do dono.
4. O administrador pode editar, concluir e excluir qualquer tarefa — mas as regras RN04 (não editar concluída) e RN06 (confirmar exclusão) continuam valendo integralmente para ele.

## Passo 10 — Encerrar

1. O usuário aciona "Sair".
2. O sistema encerra a sessão e devolve o usuário à tela de login.
3. Nenhum dado se perde: cada operação de criação, edição, conclusão ou exclusão já regravou o arquivo no momento em que aconteceu.

## 4. Mapeamento arquitetural para Git e GitHub

Segundo a webaula 2, este conteúdo deve ser produzido no Google Apresentações, então o texto abaixo é uma versão mais explicativa da mesma seção.

### 4.1 Árvore de pastas do repositório

```
task-tracker-bootcamp2/
├── README.md
├── LICENSE
├── .gitignore
├── especificacao/
│   ├── planejamento-logico.docx
│   ├── mapeamento-arquitetural.pptx
│   └── Entrega Etapa Inicial.docx
├── implementacao/
│   ├── requirements.txt
│   ├── src/
│   │   ├── app.py
│   │   ├── configuracao.py
│   │   ├── modelos/
│   │   │   ├── tarefa.py
│   │   │   └── usuario.py
│   │   ├── servicos/
│   │   │   ├── gerenciador_tarefas.py
│   │   │   ├── gerenciador_usuarios.py
│   │   │   ├── validacoes.py
│   │   │   └── relatorio.py
│   │   ├── repositorio/
│   │   │   └── repositorio_json.py
│   │   ├── templates/
│   │   │   ├── base.html
│   │   │   ├── login.html
│   │   │   ├── cadastro_usuario.html
│   │   │   ├── painel.html
│   │   │   ├── formulario_tarefa.html
│   │   │   ├── confirmar_exclusao.html
│   │   │   ├── relatorio.html
│   │   │   └── admin.html
│   │   └── estaticos/
│   │       └── estilo.css
│   ├── dados/
│   │   ├── tarefas.json
│   │   └── usuarios.json
│   ├── tests/
│   │   ├── test_validacoes.py
│   │   ├── test_gerenciar_tarefas.py
│   │   └── test_autenticacao.py
│   └── docker/
│       ├── Dockerfile
│       ├── docker-compose.yml
│       └── .dockerignore
```

## 4.2 Propósito de cada pasta e arquivo

**README.md** — é a porta de entrada do repositório e a primeira coisa que o GitHub exibe para quem abre a página do projeto. Nele vão a descrição do Task Tracker, o objetivo, as instruções de instalação e execução e o link do vídeo de apresentação.

**LICENSE** — declara sob quais condições o código pode ser usado, copiado e modificado por terceiros.

**.gitignore** — lista os arquivos e pastas que o Git deve deliberadamente ignorar, ou seja, que nunca devem entrar em um commit.

**especificacao/** — guarda toda a documentação de planejamento, ou seja, o produto desta Etapa 1.

**implementacao/** — concentra tudo o que é executável, ou seja, o produto da Etapa 2. Uma pasta própria deixa evidente a fronteira entre o que é documento e o que é programa.

**implementacao/requirements.txt** — lista as bibliotecas externas de que o projeto depende. É esse arquivo que permite que qualquer pessoa e o Docker reproduzam o ambiente exato do projeto com um único comando.

**implementacao/src/** — contém o código do app, organizado em camadas, evitando ter uma única parede de código ilegível.

**src/app.py** — é o ponto de entrada da aplicação. Ele coordena, mas não contém regras de negócio.

**src/configuracao.py** — reúne num só lugar os valores que definem o comportamento do sistema.

**src/modelos/** — define as classes que representam as entidades do sistema, fazendo um sistema orientado a objetos.

**src/servicos/** — define as regras de negócio.

**src/repositorio/** — um o único arquivo que pode ler e gravar os arquivos JSON. Se um dia eu quiser trocar o JSON por um banco SQLite, só esse arquivo precisa mudar.

**src/templates/** — guarda as páginas HTML que o usuário vê, escritas com Jinja2.

**src/estaticos/** — contém os arquivos css servidos ao navegador sem processamento

**implementacao/dados/** — armazena os arquivos JSON. Ficam numa pasta separada do código, porque são dados e não programa, e porque na Etapa 3 essa pasta será montada como volume do Docker.

**implementacao/tests/** — vai reunir os testes automatizados escritos com pytest. Os testes serão a evidência objetiva de que as regras documentadas nesta etapa foram de fato implementadas na Fase 2.

**docker/** — reúne tudo o que é necessário para empacotar e executar o sistema em contêiner, na Etapa 3.

### 4.3 Conceitos de Git e GitHub que serão aplicados na próxima etapa

Na Etapa 2 o repositório deixa de ser armazenamento de arquivos e passa a ser o registro de como o sistema foi construído. O elemento central desse registro é o commit, uma fotografia do projeto num instante, acompanhada de uma mensagem que explica o que mudou e por quê. O fluxo é sempre o mesmo, alterar arquivos, `git add` para escolher quais alterações entram, e `git commit -m "xxx"` para registrá-la definitivamente.

O ponto que pretendo respeitar é que um commit deve ter um assunto só. Vários ajustes em um commit, prejudica o entendimento cronológico, inclusive o meu daqui a duas semanas.

Para organizar o trabalho, vou usar branches. A branch main guarda apenas versões que funcionam. Cada funcionalidade nova nasce em uma branch quebrar o que já está pronto. Quando a funcionalidade estiver concluída, ela volta para a main através de um merge, feito por um Pull Request no GitHub. Mesmo trabalhando sozinho, usar Pull Request tem valor, obrigando a reler o próprio código antes de integrá-lo e deixa registrada, na página do repositório, a justificativa de cada bloco de mudança.

O `git push` é usado para enviar os commits locais ao GitHub e `git pull` para trazer o que estiver lá. Por fim, o `.gitignore` entra nesse conjunto de boas práticas: ao manter fora do controle de versão o ambiente virtual e os arquivos gerados automaticamente, ele elimina de saída uma das causas mais comuns de conflito desnecessário.

## 5. Link da apresentação em vídeo

---

<https://www.youtube.com/@pedrorebelloceub>